

An Abstract, Guarded Categorical Model of Gradual Typing

Eric Giovannini

September 2, 2025

1 Introduction

The work of Giovannini et al. [1] constructs a denotational semantics for a simple gradually-typed lambda calculus using guarded type theory. Many of the aspects of the model construction are in fact independent of the details of the particular language being modeled, e.g., the specific types and effects in the language. In this work, we seek to abstract over these details and present an abstract categorical framework for constructing models of gradual typing in guarded type theory.

With this abstract framework in hand, we then define concrete semantics for gradually-typed languages with simple effects.

1.1 Why an Abstract Model?

The model developed by Giovannini et al. suffers from the limitation that it is a single model for a quite bare-bones gradually-typed language. Real-world languages can include effects, polymorphism, dependent types, etc. The goals of an abstract model are twofold:

1. To promote a better understanding of the essence of any guarded relational semantics of gradual typing independent of the particular language features being modeled.
2. To allow for developers of languages to establish graduality for their language by instantiating our abstract framework.

1.2 Challenges

Prior work by New and Licata [2] defines a categorical model for gradual typing that serves as the basis for denotational semantics using *classical* domain theory. Their model is based on *double categories* [4]. Unfortunately, as Giovannini et al. point out, adapting this model to the guarded setting poses significant challenges, and much of their work consists of defining a version of the model that works in guarded type theory.

Inspired by the solution of Giovannini et al., we refine the notion of abstract categorical model of gradual typing into two novel classes of model suitable for the guarded setting. We call these *extensional* and *intensional* models.

1.3 Outline

This work is organized as follows.

- We begin in Section 2 with an overview of the existing abstract double-categorical semantics for gradual typing by New and Licata.
- We review the difficulties in adapting the New-Licata denotational model to the guarded setting.
- We motivate the refinement of the notion of abstract model into extensional and intensional models and present their definitions in full.

- We discuss the construction of an extensional model of gradual typing from a suitable “base” double category of effectful functions and relations.
- With the constructions in hand, we then instantiate the abstract framework to define concrete models of gradual typing capable of modeling languages with simple effects.

1.4 Ambient Guarded Metalanguage

Because guarded type theory plays a central role in the construction of the concrete model in prior work, it is natural to expect that our abstract framework should to some degree incorporate aspects of guarded type theory. To that end, we demand the following of the categories involved in the description of our abstract model:

1. There is an applicative functor $\triangleright : \mathcal{C} \rightarrow \mathcal{C}$.
2. There is a morphism $\text{next} : X \rightarrow \triangleright X$ for any object X .
3. For any morphism f of the form $\triangleright X \rightarrow X$, there is a unique morphism $\text{fix}f : 1 \rightarrow X$ such that $\text{fix}f = f \circ \text{next} \circ \text{fix}f$.

2 Double Categorical Semantics of Graduality

We begin with a review of abstract categorical models for gradual typing described in prior work. New and Licata [2] modeled the graduality and type casts for call-by-name gradual typing using *double categories*, which are defined to be categories internal to the category of categories. That is, a double category $\mathcal{C}$ consists of a category $\mathcal{C}_f$ of “objects and function morphisms” and a category $\mathcal{C}_{sq}$ of “relation morphisms and squares” with functors (reflexive relation) $r : \mathcal{C}_f \rightarrow \mathcal{C}_{sq}$ and (source and target) $s, t : \mathcal{C}_{sq} \rightarrow \mathcal{C}_f$ satisfying $sr = tr = \text{id}$ as well as a composition operation $c : \mathcal{C}_{sq} \times_{s,t} \mathcal{C}_{sq} \rightarrow \mathcal{C}_{sq}$ respecting source and target. This models an abstract notion of functions and relations. For notation, we write function morphisms as $f : A \rightarrow B$ and relation morphisms as $c : A \multimap B$ where $c \in \mathcal{C}_{sq}$ and $s(c) = A$ and $t(c) = B$. Finally, a morphism α from c to d with $s(\alpha) = f$ and $t(\alpha) = g$ is visualized as

$$\begin{array}{ccc} A & \multimap^c & B \\ f \downarrow & & \downarrow g \\ A' & \multimap_d & B' \end{array}$$

Such a morphism is thought of as an abstraction of the notion of relatedness of functions: functions take related inputs to related outputs. The composition operations and functoriality give us a notion of composition of relations as well as functions and vertical and horizontal composition of squares. In this work we will be chiefly interested in *locally thin* double categories, that is, double categories where there is at most one square for any f, c, g, d . In this case we use the notation $f \sqsubseteq_d^c g$ to mean that a square like the above exists.

New, Licata and Ahmed [3] extended the axiomatic syntax to call-by-push-value but did not analyze the structure categorically. We fill in this missing analysis now, extending the double categorical semantics given in [2] from cartesian closed categories to CBPV models.

We begin by reviewing the definition of a CBPV model:

Definition 2.1. A CBPV model $\mathcal{M}$ consists of:

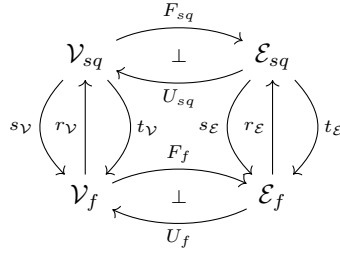
- A cartesian-closed category $\mathcal{V}$ of values and pure morphisms.
- A category $\mathcal{E}$ of computations and linear morphisms.
- Functors $F : \mathcal{V} \rightarrow \mathcal{E}$ and $U : \mathcal{E} \rightarrow \mathcal{V}$ with $F \dashv U$.
- An action of $\mathcal{V}$ on $\mathcal{E}$, i.e., a functor $\rightarrow : \mathcal{V}^{op} \times \mathcal{E} \rightarrow \mathcal{E}$ satisfying the usual coherence laws, and such that $U(A \rightarrow B)$ is isomorphic to $A \Rightarrow UB$.

A *strict homomorphism of CBPV models* from $(\mathcal{V}, \mathcal{E}, \dots)$ to $(\mathcal{V}', \mathcal{E}', \dots)$ consists of functors $G_v : \mathcal{V} \rightarrow \mathcal{V}'$ and $G_e : \mathcal{E} \rightarrow \mathcal{E}'$ that strictly preserve all CBPV constructions; see the appendix for a more detailed definition. We call this a *strict morphism* in contrast to a *lax* morphism, which only preserves CBPV constructions up to transformation.

A model of the congruence rules of the CBPV system can be given by a locally thin “double CBPV model”, which we define below:

Definition 2.2. A *double CBPV model* is a category internal to the category of CBPV models and *strict* homomorphisms of CBPV models¹.

Part of the data of a double CBPV model can be visualized as follows:



The diagram above makes it clear that there are two useful ways of viewing a double CBPV model $\mathcal{M}$. First, we can view $\mathcal{M}$ as pair of CBPV models, one forming the objects and vertical morphisms (the bottom of the diagram) and the other forming the horizontal morphisms and the squares (the top of the diagram). Alternatively, we can view $\mathcal{M}$ as a pair of double categories, one for the values (the left of the diagram), and one for the computations (the right of the diagram). For the latter viewpoint, we will often write $\mathcal{C}$ to stand for an unspecified such double category associated with a double CBPV model.

Given a double CBPV model, we can interpret the syntax of gradual typing as follows. Type precision $A \sqsubseteq A'$ is interpreted as a relation morphism $c_A : A \multimap A'$ in $\mathcal{V}_{sq}$, and term precision $\Gamma \sqsubseteq \Gamma' \vdash M \sqsubseteq M' : A \sqsubseteq A'$ is interpreted as a square $M \sqsubseteq_{U F c_A}^{\text{ct}} M'$. The fact that t, r and the composition are all given by strict CBPV homomorphisms says that all the type constructors lift to precision (monotonicity of type constructors) as well as all term constructors (congruence). Further, r and composition being strict homomorphisms implies that all type constructors strictly preserve the identity relation (identity extension) and composition.

Next, to model type casts, the New-Licata model further requires that every value relation $c : A \multimap A'$ is *left representable* by a function $u_c : A \rightarrow A'$ and every computation relation $d : B \multimap B'$ is *right representable* by a function $d_c : B' \rightarrow B$. In a locally thin double category, left and right representability are defined as follows:

Definition 2.3. A relation $R : A \multimap B$ is *left representable* by $f : A \rightarrow B$ if $f \sqsubseteq_{r(B)}^R \text{id}$ and $\text{id} \sqsubseteq_R^{r(A)} f$.

Dually, $R : A \multimap B$ is *right representable* by $g : B \rightarrow A$ if $\text{id} \sqsubseteq_{r(A)}^R g$ and $g \sqsubseteq_R^{r(B)} \text{id}$.

These squares are sufficient to model the UpL/UpR/DnL/DnR rules for casts. Additionally, since representable morphisms compose, the compositionality of casts comes for free. However, the retraction property must be added as an additional axiom to the model. To model the error being the least element, the New-Licata model adds the requirement that $\text{id} \circ ! \sqsubseteq_{r(U_B)}^{r(A)} f$ holds for all $f : \mathcal{V}(A, B)$. Finally, the dynamic type can be modeled as an arbitrary value type D with arbitrary relations $\text{Nat} \multimap D$ and $D \multimap D \multimap D$ and $D \times D \multimap D$ (or whatever basic type cases are required).

¹it may be possible to also define this as a notion of CBPV model internal to some structured 2-category of categories, but we are not aware of any such definition of an internal CBPV model

3 Adapting the Notion of Model to Guarded Type Theory

3.1 Challenges in Adapting the New-Licata Model

While the original double categorical semantics introduced in Section 2 can be satisfied with classical domain theoretic models as demonstrated by New and Licata [2], Giovannini et al. [1] show that the New-Licata model cannot be directly adapted to work in the guarded setting.

Key to understanding the issue is the fact that the equation for the dynamic type involves a later, which means that when inspecting an element of the dynamic type which is a function, an *observable step* is required in the semantics. This means that downcasts from the dynamic type may take an observable step, and thus *any* cast may take steps as it may involve a downcast from the dynamic type functorially.

The issue in adapting the New-Licata model to the guarded setting is related to the representability condition on the relations. At a high level, the problem arises as a result of a tension between *transitive reasoning* and *step-insensitivity*. Transitive reasoning is desirable because it allows us to derive the cast rules needed for graduality from the simpler, compositional versions of the rules. This is crucial because the original rules quantify over an additional, arbitrary type precision derivation and are therefore not syntax-independent. On the other hand, the graduality property, as axiomatized by the representability of type precision relations, is fundamentally a step-insensitive property. The casts, which may involve observable steps, must be related to the identity function, which does not step. The relation modeling term precision must be able to account for this mismatch.

However, Giovannini et al. show that transitivity and unrestricted step-insensitivity (plus a third minor condition) are incompatible as properties of a relation in guarded type theory (see their Theorem 4.1 for a precise statement). The solution adopted by Giovannini et al. is to work with a *lock-step error ordering* that is transitive, and then restore a small amount of step-insensitivity where needed through explicit synchronizing terms they call *perturbations*. The perturbations mimic the stepping behavior of casts, but are otherwise equivalent to the identity, a property they formalize via a notion of *weak bisimilarity*. The authors relax the definition of representability to allow for casts to be lock-step related to perturbations rather than only to the identity function. Finally, to define the denotation of term precision, the authors take the closure of the lock-step ordering under weak bisimilarity on both sides:

$$M \tilde{\sqsubseteq} N := \exists f, g. M \approx f \sqsubseteq g \approx N,$$

where $\approx$ is weak bisimilarity and $\sqsubseteq$ is the lock-step ordering. The idea is that two terms are related if they can be synchronized in some number of steps to two terms that are *lock-step* related. By separating term precision into a combination of weak bisimilarity and lock-step reasoning, and using syntactic perturbations as a means to explicitly synchronize terms as necessary, the authors achieve compositional reasoning while still accommodating the steps taken by casts.

3.2 Refining the Notion of Abstract Model

As discussed in the previous section, the original notion of double categorical model of gradual typing is not compatible with guarded type theory, where the denotational semantics involves observable steps. We are therefore led to *refine* this notion of model to one that is compatible with guarded type theory. To motivate this refinement, consider the model of Giovannini et al. [1] as reviewed in the previous section. Consider the following two key observations about their model:

1. The denotation of term precision (i.e., $\tilde{\sqsubseteq}$) is *not* transitive. This follows from their no-go theorem (Theorem 4.1), as the relation satisfies the other two properties in the theorem and so it cannot be transitive.
2. It is possible to derive the original representability rules for casts from the simplified versions using transitive reasoning.

At first glance, these two observations seem contradictory. How can one use transitivity if it does not hold? The key insight is that observations (1) and (2) take place in two distinct settings. The derivation

described in observation (2) occurs *before* the final denotation of term precision is defined. That is, in (2), the relation in question is the lock-step ordering, *not* its closure under weak bisimilarity. The lock-step ordering is transitive, and thus we can derive the required representability squares from the simplified ones via horizontal composition of squares. Recall that perturbations are involved at this point in order to keep terms in lock-step. Thus, much of the work in the model construction consists of showing that the theory of perturbations is compositional in the appropriate manner.

The existence of these two distinct settings in the concrete model motivates our refinement of the idealized abstract model of gradual typing. In particular, we divide the idealized notion of model into two classes, which we call *extensional* and *intensional* models. The formal definitions will follow, but the main idea can be understood in light of observations (1) and (2) above. An *extensional model* is an abstraction of the idea of working “up to weak bisimilarity”, i.e., extensionally. Such a model does not include transitivity of term precision, since transitive reasoning and extensional reasoning are incompatible in the guarded setting. On the other hand, an extensional model does validate the rules for casts (in their original form). Thus, we can interpret the syntax of gradual typing into an extensional model, as the graduality property boils down to the validity of the representability rules.

In contrast, an *intensional model* is an abstraction of the idea of working with a transitive, lock-step ordering, and using perturbations to synchronize terms as needed. The restricted form of step-insensitivity provided by perturbations allows us to reason up to observable steps without using weak bisimilarity. Intensional models are appealing because they allow us to apply compositional reasoning principles, e.g., deriving the original forms of the cast rules from the simplified versions. On the other hand, intensional models do not directly validate graduality, as we need to “collapse” the perturbations to define the denotation of term precision. However, from an intensional model, we can *construct* an extensional model by performing the abstract analogue of closing the lock-step ordering under weak bisimilarity.

In the remainder of this section, we define extensional and intensional models. In the following section, we discuss how to construct an extensional model.

3.3 Extensional Models of Gradual Typing

Since we lack transitivity of ordering when reasoning extensionally in guarded type theory, our weakened notion of extensional model is based on reflexive graph categories rather than double categories. This means we lose the reasoning principle of horizontal pasting of squares. We will still require a notion of *composition* of relations, to model the transitivity of type precision. We note that without horizontal pasting of squares, the notion of left/right representability of relations is not sufficient to interpret the cast rules of gradual typing. Instead, in an extensional model we work with a generalized notion of representability that corresponds to the non-compositional form of the cast rules:

Definition 3.1. Let $c : A \multimap A'$ and $f : A \rightarrow A'$ in a reflexive graph category with composition of relations. We say that c is *universally left-representable by f* if for any $c_l : A_l \multimap A$ and $c_r : A' \multimap A_r$ we have $f \sqsubseteq_{c_r}^{c c_l} \text{id}$ and $\text{id} \sqsubseteq_{c_l c}^{c_l} f$. Dually, let $d : B \multimap B'$ and $\phi : B' \rightarrow B$. We say that d is *universally right-representable by ϕ* if for any $d_l : B_l \multimap B$ and $d_r : B' \multimap B_r$ we have $\text{id} \sqsubseteq_{d_l}^{d_l d} \phi$ and $\phi \sqsubseteq_{d d_r}^{d_r} \text{id}$.

In a reflexive graph category, these notions are stronger than the definitions of left/right representability, since we can pick c_l, c_r, d_l, d_r to be the reflexive relations. In a double category these are equivalent, but the equivalence uses horizontal pasting.

Additionally, while in the presence of horizontal pasting compositionality of representable morphisms is automatic, without this principle we must require it explicitly. Finally, we explicitly add in a requirement that the type constructors are functorial on relations up to order equivalence.

In summary:

Definition 3.2. An *extensional model of gradual typing* consists of:

1. A locally thin reflexive graph internal to CBPV models.

2. Composition of value and computation relations that form a category with the reflexive relations as identity. Call these categories $\mathcal{V}_r, \mathcal{E}_r$.
3. Identity-on-objects functors $\text{up} : \mathcal{V}_r \rightarrow \mathcal{V}_f$ and $\text{dn} : \mathcal{E}_r^{\text{op}} \rightarrow \mathcal{E}_f$ such that every $\text{up } c$ universally left-represents c and every $\text{dn } d$ universally right-represents d .
4. The CBPV connectives $U, F, \times, \rightarrow$ are all *covariant* functorial on relations up to equivalence: $U(dd') \sqsubseteq\sqsubseteq U(d)U(d')$ etc.²
 - $U(dd') \sqsubseteq\sqsubseteq U(d)U(d')$
 - $F(cc') \sqsubseteq\sqsubseteq F(c)F(c')$
 - $(cc') \rightarrow (dd') \sqsubseteq\sqsubseteq (c \rightarrow d)(c' \rightarrow d')$
 - $(c_1c'_1) \times (c_2c'_2) \sqsubseteq\sqsubseteq (c_1 \times c_2)(c'_1 \times c'_2)$

where $c \sqsubseteq\sqsubseteq c'$ means $\text{id} \sqsubseteq_{c'}^c \text{id}$ and $\text{id} \sqsubseteq_c^{c'} \text{id}$.
5. A natural transformation $\mathcal{U} : 1 \Rightarrow U$ such that $\mathcal{U} \circ ! \sqsubseteq_{r(UB)}^{r(A)} f$ for any $f : A \rightarrow UB$.
6. Distinguished value type Nat with morphisms $z : \mathcal{V}(1, \text{Nat})$ and $s : \mathcal{V}(\text{Nat}, \text{Nat})$.
7. Distinguished value types D with distinguished relations $\text{Inj}_{\rightarrow} : U(D \rightarrow FD) \multimap D$ and $\text{Inj}_{\mathbb{N}} : \text{Nat} \multimap D$ and $\text{Inj}_{\times} : D \times D \multimap D$ each satisfying the retraction property $\text{dn } \text{Inj}_{\rightarrow} F(\text{up } \text{Inj}_{\rightarrow}) \sqsubseteq\sqsubseteq \text{id}$.

An extensional model of gradual typing is sufficient to interpret the syntax of a gradual cast calculus.

3.4 Intensional Models of Gradual Typing

An intensional model of gradual typing is an abstraction of working in a setting where the ordering on terms is transitive but requires terms to be in lock-step. To aid in understanding the definition of an intensional model, we break the definition into five steps, with each one building on the previous. The last step will be the definition we have in mind when we refer to an intensional model.

The starting point for an intensional model is the same as that of an idealized model of gradual typing: namely, a double CBPV model (see Definition 2.2). In particular, unlike the extensional model where we could only compose *relations* horizontally, here we also have a horizontal composition operation on squares.

As in the definition of extensional model, we also require the existence of a natural transformation $\mathcal{U} : 1 \Rightarrow U$ such that $\mathcal{U} \circ ! \sqsubseteq_{r(UB)}^{r(A)} f$ for any $f : A \rightarrow UB$, and a distinguished value type Nat with morphisms $z : \mathcal{V}(1, \text{Nat})$ and $s : \mathcal{V}(\text{Nat}, \text{Nat})$.

To model intensional behavior, we specify that the model has a natural transformation $\delta : U \Rightarrow U$. Specifically, this means there is a family of vertical morphisms $\delta_B \in \mathcal{V}_f(UB, UB)$, which is natural in B with respect to composition of vertical morphisms: for all $\phi \in \mathcal{E}_f(B_i, B_o)$ we have $U\phi \circ \delta_{B_1} = \delta_{B_2} \circ (U\phi)$. Naturality of δ furthermore implies that for any $d : B \multimap B'$, there is a square $\delta_B \sqsubseteq_{Ud}^{Ud} \delta_{B'}$.

We call a model satisfying these requirements an *intensional pre-model*. For the sake of ease of reference, we recap the definition below:

Definition 3.3. An **intensional pre-model** consists of:

- A double CBPV model, i.e., a category internal to the category of CBPV models and lax morphisms, where we require that the morphisms r, s , and t are strict.
- A natural transformation $\mathcal{U} : 1 \Rightarrow U$ such that $\mathcal{U} \circ ! \sqsubseteq_{r(UB)}^{r(A)} f$ for any $f : A \rightarrow UB$.
- A value type Nat with morphisms $z : \mathcal{V}(1, \text{Nat})$ and $s : \mathcal{V}(\text{Nat}, \text{Nat})$.
- A natural transformation $\delta : U \Rightarrow U$.

²The reflexive graph structure already requires that these functors preserve identity relations.

Lemma 3.1. *We have the following properties involving the delay maps in any intensional pre-model:*

1. *We can define a computation morphism $\delta_{FA}^* \in \mathcal{E}_f(FA, FA)$ for all A . To do so, first compose the unit $\eta_A \in \mathcal{V}_f(A, UFA)$ with the morphism $\delta_{FA} \in \mathcal{V}_f(UFA, UFA)$, and then by the adjunction we get a computation morphism $\delta_{FA}^* \in \mathcal{E}_f(FA, FA)$.*
2. *There is a square $\delta_{FA}^* \sqsubseteq_{Fc}^{Fc} \delta_{FA'}^*$ for all $c : A \multimap A$.*

Proof. See appendix. □

3.4.1 Weak Bisimilarity

The next ingredient in the definition of intensional model is *weak bisimilarity*, which formalizes the notion that two morphisms are the same “up to steps”, i.e., they differ only in that one may wait more than the other. In particular, for any pair of objects A and A' , in $\mathcal{V}_f$, we require that there is a reflexive, symmetric relation $\approx_{A,A'}$ on the hom-set $\mathcal{V}_f(A, A')$, called the *weak bisimilarity* relation. Similarly for the computation category: there is a reflexive, symmetric relation $\approx_{B,B'}$ defined on each hom-set $\mathcal{E}_f(B, B')$. Additionally, the weak bisimilarity relation should be a congruence with respect to composition: if $f \approx_{A,A'} f'$ and $g \approx_{A',A''} g'$, then $g \circ f \approx_{A,A''} g' \circ f'$, and likewise for computations.

The bisimilarity relation should also be preserved by the functors $U, F, \rightarrow$, and $\times$ e.g., if $f \approx_{A',A} g$ and $\phi \approx_{B,B'} \phi'$, then $f \rightarrow \phi \approx_{A \rightarrow B, A' \rightarrow B'} g \rightarrow \phi'$. Similarly, the two directions of the isomorphism of hom-sets $\mathcal{V}_f(A, UB) \cong \mathcal{E}_f(FA, B)$ should preserve bisimilarity.

We can state all of this compactly by requiring that the categories $\mathcal{V}_f$ and $\mathcal{E}_f$ be *enriched* over the symmetric monoidal category $\mathbf{Set}_{\approx}$, whose objects are sets equipped with a reflexive, symmetric relation and whose arrows are functions between the sets that preserve the relation. Then we require that the functors $U, F, \rightarrow$, and $\times$ are $\mathbf{Set}_{\approx}$ -enriched, and likewise $F \dashv U$ as enriched functors. Note that $\mathcal{V}_{sq}$ and $\mathcal{E}_{sq}$ are not $\mathbf{Set}_{\approx}$ -enriched, so in the functors r, s, t between $\mathcal{V}_f$ and $\mathcal{V}_{sq}$ and between $\mathcal{E}_f$ and $\mathcal{E}_{sq}$, we view $\mathcal{V}_f$ and $\mathcal{E}_f$ as ordinary categories, where we simply forget the bisimilarity relation on hom-sets.

Lastly, we require that for any computation object B , the delay map $\delta_B : UB \rightarrow UB$ (which is defined in terms of θ_B) is bisimilar to the identity: $\delta_B \approx_{UB,UB} \text{id}_{UB}$.

Definition 3.4. An *intensional pre-model with bisimilarity* is an intensional pre-model $\mathcal{M}$ such that:

- The categories $\mathcal{V}_f$ and $\mathcal{E}_f$ are enriched over $\mathbf{Set}_{\approx}$.
- The functors $U, F, \rightarrow$, and $\times$ are $\mathbf{Set}_{\approx}$ -enriched, and the adjunction $F \dashv U$ is an enriched adjunction.
- For each B , the delay map δ_B is bisimilar to the identity $\delta_B \approx_{UB,UB} \text{id}_{UB}$.

If $\mathcal{M}$ is an intensional pre-model with bisimilarity, it will sometimes be useful to have a concise definition describing the value objects, morphisms, relations, and squares as a coherent unit, and likewise for the computations. To do this, we introduce some terminology:

Definition 3.5. A *bisimilarity reflexive-graph* $\mathcal{C}$ consists of:

- A category $\mathcal{C}_f$ of objects and vertical morphisms, enriched in $\mathbf{Set}_{\approx}$.
- A thin category $\mathcal{C}_{sq}$ of horizontal morphisms and squares.
- A functor $r : |\mathcal{C}_f| \rightarrow \mathcal{C}_{sq}$ where $|\mathcal{C}_f|$ is the underlying ordinary category of $\mathcal{C}_f$.
- Functors $s, t : \mathcal{C}_{sq} \rightarrow |\mathcal{C}_f|$ such that $sr = tr = \text{id}$.

A *functor* $G : \mathcal{C} \rightarrow \mathcal{C}'$ of bisimilarity reflexive-graphs consists of:

- A $\mathbf{Set}_{\approx}$ -enriched functor $G_f : \mathcal{C}_f \rightarrow \mathcal{C}'_f$.
- A functor $G_{sq} : \mathcal{C}_{sq} \rightarrow \mathcal{C}'_{sq}$.
- Source and target are preserved: $s' \circ G_{sq} = G_f \circ s$, and $t' \circ G_{sq} = G_f \circ t$.

- Reflexivity is preserved: $r' \circ G_f = G_{sq} \circ r$.

Let $G, H : \mathcal{C} \rightarrow \mathcal{C}'$ be functors of bisimilarity reflexive-graphs. A *natural transformation* $\alpha : G \rightarrow H$ consists of:

- A $\mathbf{Set}_{\approx}$ -enriched natural transformation $\alpha^f : G_f \rightarrow H_f$.
- A natural transformation $\alpha^{sq} : G_{sq} \rightarrow H_{sq}$.

A *bisimilarity double-category* is a bisimilarity reflexive graph that additionally has a “horizontal composition” functor $\circ : \mathcal{C}_{sq} \times_{s=t} \mathcal{C}_{sq} \rightarrow \mathcal{C}_{sq}$ where $\mathcal{C}_{sq} \times_{s=t} \mathcal{C}_{sq}$ is the following pullback:

$$\begin{array}{ccc} \mathcal{C}_{sq} \times_{s=t} \mathcal{C}_{sq} & \longrightarrow & \mathcal{C}_{sq} \\ \downarrow & \lrcorner & \downarrow \\ \mathcal{C}_{sq} & \longrightarrow & \mathcal{C}_f \end{array}$$

We additionally require that composition is well-behaved with respect to the source and target functors, and satisfies associativity and left and right unit laws, as in an ordinary double category.

A (*lax*) *functor of bisimilarity double-categories* is defined analogously to a functor of bisimilarity reflexive-graphs, except we additionally require that the functor preserves composition of horizontal morphisms up to a square:

$$\begin{array}{ccccc} GX_1 & \xrightarrow{GR} & GX_2 & \xrightarrow{GR'} & GX_3 \\ \parallel & & & & \parallel \\ GX_1 & \xrightarrow{G(R \circ R')} & & & GX_3 \end{array}$$

Bisimilarity double-categories, functors between them, and natural transformations of such functors form a 2-category, which we denote $\mathbf{DbCat}_{\approx}$:

Lemma 3.2. *There is a 2-category $\mathbf{DbCat}_{\approx}$ whose objects are bisimilarity double-categories, whose 1-cells are functors between bisimilarity double-categories, and whose 2-cells are natural transformations between such functors.*

Proof. See appendix. □

With these definitions in hand, we can give an alternative definition of an intensional pre-model with bisimilarity:

Definition 3.6. An intensional pre-model with bisimilarity consists of the following diagram in $\mathbf{DbCat}_{\approx}$:

$$\begin{array}{ccc} & F & \\ \swarrow & \downarrow \perp & \searrow \\ \mathcal{V} & \xleftarrow{U} & \mathcal{E} \\ \swarrow & \downarrow \delta & \searrow \\ & U & \\ \swarrow & \uparrow \mathcal{U} & \searrow \\ & \Delta(1_{\mathcal{V}}) & \end{array}$$

Along with a functor of bisimilarity double-categories $\rightarrow : \mathcal{V}^{op} \times \mathcal{E} \rightarrow \mathcal{E}$ satisfying $U(A \rightarrow B) \cong A \Rightarrow UB$

Which additionally satisfies:

-

3.4.2 Perturbations

We next focus on the specification of perturbations in a pre-model with bisimilarity. To begin, let $\mathcal{C}$ be a bisimilarity double-category. For an object X of $\mathcal{C}$, a *semantic perturbation* of X is an endomorphism $f \in \mathcal{C}_f(X, X)$ such that $f \approx_{X, X} \text{id}_X$. The collection of semantic perturbations on X forms a monoid under composition, with the identity element being the identity id_X . We write $\text{Endo}_{\approx}(X)$ to refer to this monoid.

For every object $X \in \mathcal{C}_f$, we require that there is a monoid M_X of *syntactic perturbations* of X and a monoid homomorphism $i_X : M_X \rightarrow \text{Endo}_{\approx}(X)$ that interprets these as semantic perturbations.

The prototypical semantic perturbation for a computation object B is the delay map $\delta_B : UB \rightarrow UB$. We require that for any computation object B , the monoid M_{UB} contains an element m such that $i_{UB}(m) = \delta_B$.

We require that the functors $\times, \overset{k}{\times}, \rightarrow, \overset{k}{\rightarrow}, U, \overset{k}{U}, F, \overset{k}{F}$ all preserve perturbations. As in the prior work of Giovannini et al., we also require that the perturbations interact nicely with the relation morphisms. Namely, if $R : X \multimap X'$ is a relation morphism, we require there to be two homomorphisms of monoids $\text{push}_R : M_X \rightarrow M_{X'}$ and $\text{pull}_R : M_{X'} \rightarrow M_X$. The authors call this the *push-pull property*.

If $\mathcal{M}$ is a pre-model with bisimilarity that additionally satisfies the above properties, we say that $\mathcal{M}$ is a *pre-model with perturbations*. To summarize:

Definition 3.7. A *pre-model with perturbations* $\mathcal{M}$ is a pre-model with bisimilarity, such that the following additional requirements are met:

1. For each object (value or computation) X , there is a monoid M_X and a homomorphism of monoids $i_X : M_X \rightarrow \text{Endo}_{\approx}(X)$.
2. For all B , there is an element m such that $i_{UB}(m) = \delta_B$.
3. The functors $\times, \overset{k}{\times}, \rightarrow, \overset{k}{\rightarrow}, U, \overset{k}{U}, F, \overset{k}{F}$ preserve perturbations.
4. The push-pull property holds for all $c : A \multimap A'$ and all $d : B \multimap B'$.

3.5 Representable Relations

Now we turn to the notion of representability of horizontal morphisms, which allows us to specify the behavior of casts. As in the case of an idealized abstract model, we want that each value horizontal morphism c is left-representable by an embedding morphism e_c and Fc is right-representable by a projection morphism p_c , and dually for computation horizontal morphisms d . However, since casts take observable steps, we must insert perturbations on the side opposite the cast to keep both sides in lock-step. This is because the squares in an intensional model are intended to model a strong error ordering between vertical morphisms in which both sides must have the same stepping behavior. Following Giovannini et al., we call this weakened notion of representability up-to an explicit perturbation *quasi-representability*.

In this section, we begin with the notion of intensional pre-model with perturbations as defined in the previous section, and provide the additional conditions that axiomatize the behavior of casts. The precise definitions are as follows.

First, let $\mathcal{M}$ be any double category with a notion of perturbations, i.e., for any object X in $\mathcal{M}$ there is a monoid M_X with a monoid homomorphism into the endomorphisms on X .

Definition 3.8. Let R be a horizontal morphism in $\mathcal{M}$ between objects X and Y . We say that R is *quasi-left-representable* by a vertical morphism f in $\mathcal{M}(X, Y)$ if there are perturbations $\delta_R^{l,e} \in M_X$ and $\delta_R^{r,e} \in M_Y$ such that there is a square $\text{UPL} : f \sqsubseteq_{r(Y)}^R i(\delta_R^{r,e})$ and a square $\text{UPR} : i(\delta_R^{l,e}) \sqsubseteq_R^{r(X)} f$.

$$\begin{array}{ccc}
 X & \xrightarrow{R} & Y \\
 f \downarrow & & \downarrow i(\delta_R^{r,e}) \\
 Y & \xrightarrow{r(Y)} & Y
 \end{array}
 \quad
 \begin{array}{ccc}
 X & \xrightarrow{r(X)} & X \\
 i(\delta_R^{l,e}) \downarrow & & \downarrow f \\
 X & \xrightarrow{R} & Y
 \end{array}$$

Definition 3.9. Let R be a horizontal morphism between X and Y . We say that R is *quasi-right-representable* by $f \in \mathcal{M}(Y, X)$ if there exist perturbations $\delta_R^{l,p} \in M_X$ and $\delta_R^{r,p} \in M_Y$ such that we have a square DNR : $i(\delta_R^{l,p}) \sqsubseteq_{r(X)}^R f$ and a square DNL : $f \sqsubseteq_R^{r(Y)} i(\delta_R^{r,p})$.

We can now define a notion of an intensional model where the horizontal morphisms are quasi-representable:

Definition 3.10. Let $\mathcal{M}$ be an intensional pre-model with perturbations. We say that $\mathcal{M}$ is a *pre-model with representability* if:

- Every value horizontal morphism $c : A \multimap A'$ is quasi-left-representable in the double category $\mathcal{V}$. That is, there is a morphism $\text{up } c \in \mathcal{V}(A, A')$ that quasi-left-represents c .
- Every computation horizontal morphism $d : B \multimap B'$ is quasi-right-representable in $\mathcal{E}$. That is, there is a morphism $\text{dn } d \in \mathcal{E}(B', B)$ that quasi-right-represents d .

3.6 The Dynamic Type

We now discuss how to specify the denotation of the dynamic type in an intensional model. It is important to note that it makes sense to axiomatize the dynamic type already in a base intensional pre-model. That is, we do not require the notions of weak bisimilarity, perturbations, or representability of horizontal morphisms to define the denotation of the dynamic type.

Let $\mathcal{M}$ be an intensional pre-model, a pre-model with bisimilarity, a pre-model with perturbations, or a pre-model with representability. To interpret the dynamic type, we require that there is a distinguished value object $D \in \mathcal{V}$, such that for every value object A , there is a distinguished horizontal morphism $\text{inj}_A : A \multimap D$.

Now we can state the definition of an intensional model:

Definition 3.11. An *intensional model of gradual typing* is an intensional pre-model with representability that furthermore has an interpretation of the dynamic type.

4 Constructing a Guarded Model of Gradual Typing

In this section, we discuss how to *construct* an abstract model of gradual typing via a step-by-step process. Recall that the ultimate goal is to build an extensional model of gradual typing (Definition 3.2), as such a model contains the structure necessary to interpret the syntax and precision rules of a gradual cast calculus.

Although the definition of an extensional model involves a significant amount of data, it is not necessary to create an instance of an extensional model from scratch. Much of the difficult work can be carried out abstractly. In particular, it is possible to construct an extensional model from an intensional model using the abstract analogue of the bisimilarity-closure construction in the work of Giovannini et al. Moreover, much of the construction of an intensional model can also be carried out abstractly.

We take as a starting point an intensional pre-model with bisimilarity (Definition 3.4).

4.1 Adding Perturbations

Suppose we are given an intensional pre-model with bisimilarity $\mathcal{M}$. Our goal is to construct a model $\mathcal{M}'$ with perturbations, as in Definition 3.7.

The summary of the construction is as follows:

- A value object in $\mathcal{M}'$ consists of a value object A of $\mathcal{M}$ paired with a monoid M_A of syntactic perturbations, and a homomorphism of monoids $i_A : M_A \rightarrow \text{Endo}_{\approx}(A)$ interpreting the perturbations as semantic perturbations. Dually, a computation object of $\mathcal{M}'$ consists of a computation object B of $\mathcal{M}$ paired with a monoid M_B , along with a monoid homomorphism $i_B : M_B \rightarrow \text{Endo}_{\approx}(B)$.
- The value and computation vertical morphisms in $\mathcal{M}'$ are defined simply as those of $\mathcal{M}$; they do not interact with the perturbations.

- A value relation $c : (A, M_A, i_A) \multimap (A', M_{A'}, i_{A'})$ in $\mathcal{M}'$ is defined to be a value relation $c : A \multimap A'$ in $\mathcal{M}$ that satisfies the push-pull property with respect to the monoids M_A and $M_{A'}$. Dually, a computation relation $d : (B, M_B, i_B) \multimap (B', M_{B'}, i_{B'})$ in $\mathcal{M}'$ is defined to be a computation relation in $\mathcal{M}$ that satisfies the push-pull property with respect to M_B and $M_{B'}$.
- Lastly, value and computation squares in $\mathcal{M}'$ are defined to be the same as those in $\mathcal{M}$.

4.2 Adding Representability

4.3 The Dynamic Type

Lastly, suppose we are given an intensional pre-model with representability $\mathcal{M}$, along with an object D with distinguished relations $\text{Inj}_{\mathbb{N}}, \text{Inj}_{\times}, \text{Inj}_{\rightarrow}$. We construct a model for which each value object A has a corresponding horizontal morphism $\text{inj}_c : A \multimap D$.

4.4 Extracting an Extensional Model: The Bisimilarity-Closure Construction

Lastly, we can obtain an extensional model from an intensional model using the abstract analogue of the bisimilarity-closure definition in the work of Giovannini et al. In particular, let $\mathcal{M}$ be an intensional model. We define an extensional model $\mathcal{M}'$ as follows. The objects and vertical/horizontal morphisms are the same as those in $\mathcal{M}$; it is only the definition of squares that differs.

Specifically, let $f : X_i \rightarrow X_o$ and $g : X'_i \rightarrow X'_o$ be vertical morphisms (of either value or computation types). Let $R_i : X_i \multimap X'_i$ and $R_o : X_o \multimap X'_o$ be horizontal morphisms. We define

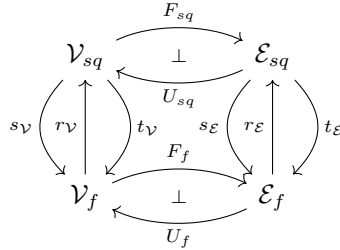
$$f \sqsubseteq_{R_o}^{R_i} g := \exists f', g'. f \approx_{X_i, X_o} f' \sqsubseteq_{R_o}^{R_i} g' \approx_{X'_i, X'_o} g.$$

It remains to verify that the model satisfies the requirements of an extensional model. The basic idea is that the perturbations in the intensional squares are all bisimilar to the identity function, and hence they disappear when we perform the above construction.

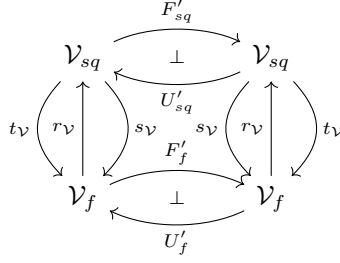
5 Constructions on Abstract Models

In this section, we explain how to augment a model of gradual typing with additional effects presented in the form of an adjunction on the value or computation categories. We assume the minimum amount of data given, i.e., an intensional pre-model with bisimilarity $\mathcal{M}$ (see Definition 3.4), along with a suitable value object interpreting the dynamic type.

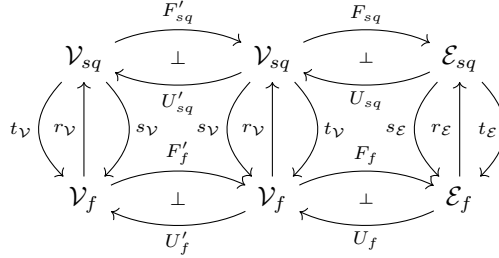
Recall that we can visualize (part of) the underlying double CBPV model of $\mathcal{M}$ as follows:



Now suppose we are given a pair of adjunctions $F'_f \dashv U'_f$ and $F'_{sq} \dashv U'_{sq}$, of the form



We seek conditions under which the compositions of these adjunctions forms a double CBPV model with bisimilarity of the form



6 Instantiating the Abstract Framework

Now that we have demonstrated how to construct a guarded model of gradual typing, we instantiate our framework with several examples of concrete gradually-typed languages.

6.1 A Model for the Simple Gradually-Typed Lambda Calculus

We begin with the gradually-typed lambda calculus as described in Section 2 of the paper of Giovannini et al. [1]. Following their construction, we define an intensional pre-model with bisimilarity for this language as follows:

- Value objects are *predomains* — sets A equipped with a partial ordering relation $\sqsubseteq_A$ and a reflexive, symmetric relation $\approx_A$.
- Value vertical morphisms are *predomain morphisms*. A predomain morphism between A_i and A_o is a function $f : A_i \rightarrow A_o$ between the underlying sets that preserves the ordering and bisimilarity relations.
- Value horizontal morphisms are *predomain relations*. A predomain relation c between A and A' is a relation between the underlying sets that is closed under the predomain ordering relations on both sides. Composition of predomain relations is given by the usual relational composition.
- A value square specifies the usual logical relations relationship between predomain morphisms: $f \sqsubseteq_{c_o}^{c_i} g$ means that for any $x \ c \ y$, we have $f(x) \ c_o \ g(y)$.
- Computation objects are *error domains* — predomains B that additionally have a least element $\mathcal{U}_B \sqsubseteq b$, and a morphism of predomains $\theta : \triangleright B \rightarrow B$ such that $\delta_B := \theta_B \circ \text{next}$ is bisimilar to the identity morphism.
- Computation vertical morphisms are *error domain morphisms*
- Computation horizontal morphisms are *error domain relations*. An error domain relation $d : B \multimap B'$ consists of a relation between the underlying predomains that additionally respects the error and θ elements. Composition of error domain relations d and d' is not given by composition of the underlying predomain relations; instead, it is defined inductively as the smallest error domain relation containing the composition of the underlying predomain relations.

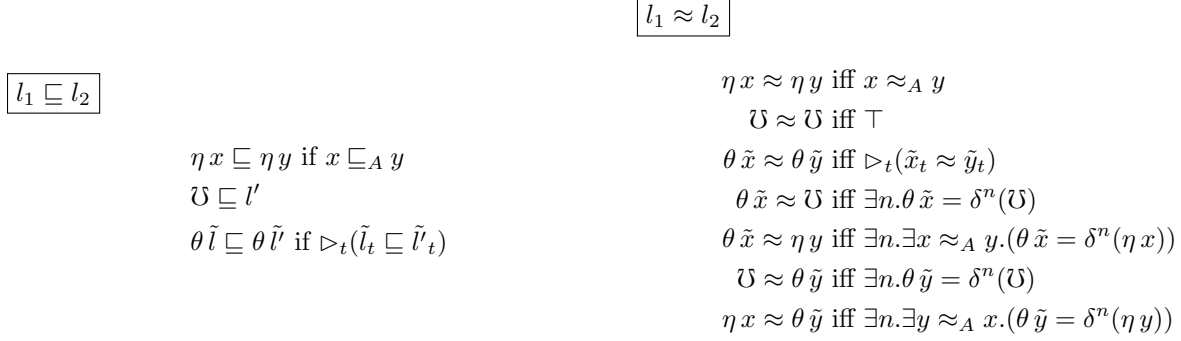


Figure 1: The lock-step error ordering and weak bisimilarity relations on FA .

- A computation square $\phi \sqsubseteq_{d'}^d \phi'$ is defined analogously to a value square.

The functor U from error domains to predomains simply forgets the error element and the θ map. The functor F from predomains to error domains takes a predomain A to the *free error domain* on A . The underlying predomain of FA is defined as the unique solution to the guarded recursive type equation

$$U(FA) \cong A + 1 + \triangleright U(FA),$$

where we name the constructors $\eta : A \rightarrow U(FA)$, $\mathcal{U} : UFA$, and $\theta : \triangleright U(FA) \rightarrow U(FA)$.

The ordering relation on FA is the *lock-step* error ordering defined in Figure 1, which also defines the weak bisimilarity relation on FA .

Given two predomains A and A' , we can form their product $A \times A'$, where the partial ordering and bisimilarity relations are the obvious extensions to the product of the underlying sets. Given predomains A_i and A_o , the set of morphisms of predomains from A_i to A_o also forms a predomain, denoted $A_i \Rightarrow A_o$, with the ordering and bisimilarity relations defined in the obvious way, e.g., $f \approx g := \forall x \approx y. f(x) \approx g(y)$.

In particular, when the codomain is of the form $U(B)$ for some error domain B , then the predomain $A \Rightarrow UB$ inherits the structure of an error domain from that of B . The error element is the constant function returning $\mathcal{U}_B$, and the θ map takes a later-morphism $\tilde{f}$ and returns the morphism $\lambda x. \theta_B((\triangleright \tilde{f})x)$ using the action of $\triangleright$ on morphisms.

It is straightforward to verify that the functors $\times, \rightarrow, U$, and F extend to relations and squares.

Lastly, to define the denotation of the dynamic type, we first construct a predomain $|D|$ satisfying

$$|D| \cong \mathbb{N} + |D| \times |D| + U(D \rightarrow FD),$$

where the functors in question are understood to be operating on the categories of predomains and error domains as appropriate. In particular, $\triangleright$ acts on predomains: for a predomain A , the ordering and bisimilarity relations on $\triangleright A$ are defined in the obvious manner — two elements of $\triangleright A$ are related if one time step later, they are related in A . Having defined the predomain for the dyanmic type, we then define the monoid of perturbations for the dynamic type by least-fixpoint in the category of monoids as

$$M_D \cong (M_D \oplus M_D) \oplus (\mathbb{N} \oplus M_D^{\text{op}} \oplus \mathbb{N} \oplus M_D),$$

and the interpretation i_D of the perturbations as endomorphisms of $|D|$ is defined straightfowardly. The resulting triple of $(|D|, M_D, i_D)$ is taken as the denotation of the dynamic type.

6.2 New Models From Scratch

Next, we discuss how we can build a new model of gradual typing from scratch given a collection of effects presented algebraically. Our goal will be to generalize the notion of error domain to accommodate these effects.

More concretely, let $\Sigma : \text{PreDom} \rightarrow \text{PreDom}$ be an endofunctor on the category of predomains (see the previous section for the definition of predomain). The purpose of Σ is to represent a signature of effects. First, to model gradual typing we need to ensure that signature has cases for errors and computational steps. To do this systematically, we define $\hat{\Sigma} := \Sigma \uplus \Delta(1) \uplus \text{Id}$, where $\uplus$ is the coproduct of functors, and $\Delta(1)$ is the constant functor returning the terminal predomain 1. We will then be interested in algebras of $\hat{\Sigma} \circ \triangleright$. Here, we note that this $\triangleright$ is operating on predomains. Furthermore, this definition implies that all occurrences of the input X are guarded by a $\triangleright$. A further generalization would be to consider algebras of a functor where some occurrences of X are unguarded and others are guarded. In this case, we would need to assume more about our given functor Σ , e.g., that it is presented as a *polynomial* endofunctor; we will pursue this in the subsequent section.

More concretely, an algebra of the functor $\hat{\Sigma} \circ \triangleright$ consists of a predomain $|B|$ along with a morphism of predomains $\text{alg}_B : \hat{\Sigma}(\triangleright|B|) \rightarrow |B|$. Note that $\hat{\Sigma}(\triangleright|B|) = \Sigma(\triangleright|B|) \uplus 1 \uplus (\triangleright|B|)$. Thus, an algebra map alg_B induces three morphisms of predomains; we denote these by $\text{op}_B : \Sigma(\triangleright|B|) \rightarrow |B|$, $\text{U}_B : 1 \rightarrow |B|$, and $\theta_B : \triangleright|B| \rightarrow |B|$. Notice also that, by definition of morphism of predomains, these maps preserve the ordering and bisimilarity relations of the relevant predomains.

We can now define the object that will serve as the analogue of error domains in our construction:

Definition 6.1. Let $\Sigma : \text{PreDom} \rightarrow \text{PreDom}$ be a functor. A Σ -domain is an algebra of the functor $\hat{\Sigma} \circ \triangleright$. Furthermore, the element $\text{U}_B : 1 \rightarrow B$ has the property that $\text{U}_B \sqsubseteq_B x$ for all $x \in B$.

Example 1. For $\Sigma := \Delta(0)$, where 0 is the empty predomain, we recover the original notion of error domain from Section 6.1.

As with ordinary error domains, there are corresponding notions of morphisms, relations, and squares of Σ -domains:

Definition 6.2. Fix Σ as above, and let B and B' be Σ -domains. A *morphism of Σ -domains* from B to B' is a morphism ϕ of the underlying predomains that preserves the algebraic structure, i.e., $\text{alg}_{B'} \circ (\hat{\Sigma} \circ \triangleright)(\phi) = \phi \circ \text{alg}_B$. Spelling this out concretely, we have in particular that (1) $\phi \circ \text{op}_B = \text{op}_{B'} \circ ((\hat{\Sigma} \circ \triangleright)(\phi))$, (2) $\phi(\text{U}_B) = \text{U}_{B'}$, and (3) $\phi \circ \theta_B = \theta_{B'} \circ (\triangleright\phi)$.

A *relation of Σ -domains* $d : B \multimap B'$ is a relation $|d|$ of the underlying predomains that is a congruence with respect to the algebraic structure, i.e., if $x \hat{\Sigma}(\triangleright|d|) y$, then $\text{alg}_B(x) |d| \text{alg}_{B'}(y)$. (Here, we have used the actions of the predomain functor $\hat{\Sigma}$ and the predomain functor $\triangleright$ on predomain relations.) Spelling out the above requirement, we have in particular (1) If $x (\Sigma(\triangleright|d|)) y$, then $\text{op}_B(x) |d| \text{op}_{B'}(y)$. (2) $\text{U}_B |d| \text{U}_{B'}$, and (3) If $\triangleright_t(\tilde{x}_t |d| \tilde{y}_t)$, then $\theta_B(\tilde{x}) |d| \theta_{B'}(\tilde{y})$.

A *square of Σ -domains* $\phi \sqsubseteq_{d'}^d \phi'$ is defined analogously to a square of predomains.

Remark 1. The requirement that U_B be related to $\text{U}_{B'}$ in the definition of relation may seem weaker than the corresponding requirement in the definition of a relation of error domains in the previous section, where we required that $\text{U}_B |d| y$ for *all* $y \in B'$. However, the former implies the latter using the fact that predomain relations are closed under the ordering on both sides and that $\text{U}_{B'} \sqsubseteq y$ for all y .

Now we proceed to describe an intensional pre-model with bisimilarity based on an adjunction between predomains and Σ -domains. More specifically:

- As in Section 6.1, the value objects, vertical morphisms, horizontal morphisms, and squares are respectively predomains, morphisms of predomains, predomain relations, and predomain squares.
- The computation objects are Σ -domains as defined above. Vertical morphisms, horizontal morphisms, and squares are respectively morphisms, relations, and squares of Σ -domains, as defined above.

Observe that the predomain $A \Rightarrow UB$ consisting of predomain morphisms $A \rightarrow UB$ inherits the Σ -domain structure of B .

Next, we define the adjunction $F \dashv U$ between predomains and Σ -domains. The functor U simply forgets the additional algebraic structure. The functor F constructs, for a predomain A , the free Σ -domain on A .

$\boxed{l_1 \sqsubseteq l_2}$	$\begin{aligned} & \text{Ret } x \approx \text{Ret } y \text{ iff } x \approx_A y \\ & \quad \quad \quad \mathcal{U} \approx \mathcal{U} \text{ iff } \top \\ & \theta \tilde{x} \approx \theta \tilde{y} \text{ iff } \triangleright_t(\tilde{x}_t \approx \tilde{y}_t) \\ & \text{Op } x \approx \text{Op } y \text{ if } x \Sigma(\triangleright \approx) y \\ & \quad \quad \quad \theta \tilde{x} \approx \mathcal{U} \text{ iff } \exists n. \theta \tilde{x} = \delta^n(\mathcal{U}) \\ & \quad \quad \quad \theta \tilde{x} \approx \eta y \text{ iff } \exists n. \exists x \approx_A y. (\theta \tilde{x} = \delta^n(\eta x)) \\ & \quad \quad \quad \mathcal{U} \approx \theta \tilde{y} \text{ iff } \exists n. \theta \tilde{y} = \delta^n(\mathcal{U}) \\ & \quad \quad \quad \eta x \approx \theta \tilde{y} \text{ iff } \exists n. \exists y \approx_A x. (\theta \tilde{y} = \delta^n(\eta y)) \end{aligned}$
$\begin{aligned} & \text{Ret } x \sqsubseteq \text{Ret } y \text{ if } x \sqsubseteq_A y \\ & \text{Op } x \sqsubseteq \text{Op } y \text{ if } x \Sigma(\triangleright \sqsubseteq) y \\ & \mathcal{U}_\Sigma \sqsubseteq l' \\ & \theta \tilde{l} \sqsubseteq \theta \tilde{l}' \text{ if } \triangleright_t(\tilde{l}_t \sqsubseteq \tilde{l}'_t) \end{aligned}$	

Figure 2: The lock-step error ordering and weak bisimilarity relations on $\text{GT}_\Sigma(A)$.

The underlying set is defined by guarded recursion as the type with the following constructors:

$$\begin{aligned}
\text{GT}_\Sigma(A) &:= \text{Ret} : A \rightarrow \text{GT}_\Sigma(A) \\
&| \text{Op} : \Sigma(\triangleright \text{GT}_\Sigma(A)) \rightarrow \text{GT}_\Sigma(A) & | \mathcal{U}_\Sigma : \text{GT}_\Sigma(A) \\
&| \theta_\Sigma : \triangleright \text{GT}_\Sigma(A) \rightarrow \text{GT}_\Sigma(A)
\end{aligned}$$

The ordering and bisimilarity relations on $\text{GT}_\Sigma(A)$ are a generalization of the lock-step error ordering and weak bisimilarity from the previous section. The full definitions are given in Figure 2. Notice in the cases when both sides are a Op , we pass the relation being defined to Σ , but under a later. Formally, since Σ is a functor on *predomains*, what we are actually doing is applying the guarded-recursive assumption that a predomain structure for the set $\text{GT}_\Sigma(A)$ exists later. Alternatively, instead of applying Σ to a predomain, one can think of this as simply applying its action on the ordering or bisimilarity relations to the ordering/bisimilarity relation that we assume exists later.

We observe that $\text{GT}_\Sigma(A)$ is the free Σ -domain on A : the set of morphisms of predomains $A \rightarrow UB$, where B is a Σ -domain, is in bijection with morphisms of Σ -domains $\text{GT}_\Sigma(A) \rightarrow B$.

Additionally, we define the action of the functor F on predomain relations $c : A \multimap A'$ as the heterogeneous version of the above lock-step ordering.

6.3 Generalization: Allowing for Non-Guarded Occurrences

In the previous section, we required that all recursive occurrences of the Σ -domain were guarded by a later. We can generalize this situation to allow for non-guarded occurrences, provided they are positive.

6.4 Example: State

7 Discussion

7.1 Future Work

Giovannini et. al have contributed an Agda mechanization of most of the concrete model construction described in their paper. It would be interesting to mechanize the abstract model we have formulated in this work and instantiate it to obtain the concrete model they constructed as well as models for languages with other effects as described in this work.

References

- [1] Eric Giovannini, Tingting Ding, and Max S. New. Denotational semantics of gradual typing using synthetic guarded domain theory. *Proc. ACM Program. Lang.*, 9(POPL), January 2025.
- [2] Max S. New and Daniel R. Licata. Call-by-name gradual type theory. In *Formal Structures for Computation and Deduction, Oxford England*, 2018.
- [3] Max S. New, Daniel R. Licata, and Amal Ahmed. Gradual type theory. *Proceedings of the ACM on Programming Languages (PACMPL)*, 3(POPL), jan 2019.
- [4] Michael Shulman. Framed bicategories and monoidal fibrations. *Theory and Applications of Categories*, 20, 06 2007.

A Call-by-push-value

A.1 Morphisms of CBPV Models

There are two relevant notions of *morphism* of CBPV models: *strict* and *lax*. Given call-by-push-value models $\mathcal{M}_1 = (\mathcal{V}_1, \mathcal{E}_1, \rightarrow_1, U_1, F_1)$ and $\mathcal{M}_2 = (\mathcal{V}_2, \mathcal{E}_2, \rightarrow_2, U_2, F_2)$, A *strict* morphism G from $\mathcal{M}_1$ to $\mathcal{M}_2$ is given by a pair of functors $G_{\mathcal{V}} : \mathcal{V}_1 \rightarrow \mathcal{V}_2$ and $G_{\mathcal{E}} : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ that strictly preserve the constructors:

1. $G_{\mathcal{E}} \circ F_1 = F_2 \circ G_{\mathcal{V}}$
2. $G_{\mathcal{V}} \circ U_1 = U_2 \circ G_{\mathcal{E}}$
3. $G_{\mathcal{E}}(A \rightarrow_1 B) = G_{\mathcal{V}}(A) \rightarrow_2 G_{\mathcal{E}}(B)$
4. $G_{\mathcal{V}}(A_1 \times_1 A_2) = G_{\mathcal{V}}(A_1) \times_2 G_{\mathcal{V}}(A_2)$
5. $G_{\mathcal{V}}(1_1) = 1_2$

As well as strictly preserving the corresponding universal morphisms and coherence isomorphisms.

A lax morphism instead preserves these only up to transformation

1. $G_{\mathcal{E}} \circ F_1 \Rightarrow F_2 \circ G_{\mathcal{V}}$
2. $G_{\mathcal{V}} \circ U_1 \Rightarrow U_2 \circ G_{\mathcal{E}}$
3. $G_{\mathcal{E}}(A \rightarrow_1 B) \Rightarrow G_{\mathcal{V}}(A) \rightarrow_2 G_{\mathcal{E}}(B)$
4. $G_{\mathcal{V}}(A_1 \times_1 A_2) \Rightarrow G_{\mathcal{V}}(A_1) \times_2 G_{\mathcal{V}}(A_2)$
5. $G_{\mathcal{V}}(1_1) \Rightarrow 1_2$

Additionally a lax morphism should have a relationship between these transformations and the universal morphisms, but we will only consider lax morphisms of thin categories, where such conditions hold trivially.

A.2 Kleisli Actions of CBPV Type Constructors

In CBPV models, all the type constructors are interpreted as functors:

1. $\rightarrow : \mathcal{V}^{\text{op}} \times \mathcal{E} \rightarrow \mathcal{E}$
2. $\times : \mathcal{V} \times \mathcal{V} \rightarrow \mathcal{V}$
3. $F : \mathcal{V} \rightarrow \mathcal{E}$
4. $U : \mathcal{E} \rightarrow \mathcal{V}$

That is, they all have functorial actions on *pure* morphisms of value types and *linear* morphisms of computation types. We use these functorial actions extensively in the construction of casts and their corresponding perturbations. But when defining downcasts of value types and upcasts of computation types, we additionally need a second functorial action of these categories: functoriality in *impure* morphisms of value types and *non-linear* morphisms of computation types. These notions of morphism are given by the *Kleisli* categories $\mathcal{V}_k$ and $\mathcal{E}_k$ which have value types and computation types as objects but morphisms are defined as

$$\mathcal{V}_k(A, A') = \mathcal{E}(FA, FA')$$

$$\mathcal{E}_k(B, B') = \mathcal{V}(UB, UB')$$

with composition given by composition in $\mathcal{E}/\mathcal{V}$. That is we need to define a second functorial action, that agrees with the above on objects for these Kleisli categories:

1. $\xrightarrow{k} : \mathcal{V}_k^{\text{op}} \square \mathcal{E}_k \rightarrow \mathcal{E}_k$
2. $\times^k : \mathcal{V}_k \square \mathcal{V}_k \rightarrow \mathcal{V}_k$

3. $\overset{k}{F}: \mathcal{V}_k \rightarrow \mathcal{E}_k$
4. $\overset{k}{U}: \mathcal{E}_k \rightarrow \mathcal{V}_k$

Note that rather than the product of categories we use the “funny tensor product” $\square$. This is because the action on impure/non-linear morphisms for $\overset{k}{\rightarrow} / \overset{k}{\times}$ do not satisfy “joint functoriality” but instead only “separate functoriality”, meaning we give rather than an action on morphisms in both categories simultaneously instead an action on each argument categories morphisms with the object in the other category fixed. The existence of these functorial actions for $\overset{k}{\rightarrow}$ and $\overset{k}{\times}$ is reliant on the *strength* of the adjunction. We describe them using the internal language of CBPV in order to more easily verify their existence/functoriality:

1. For $\overset{k}{\rightarrow}$ we define for $\phi : \mathcal{E}(FA, FA')$ and $B \in \mathcal{E}$ the morphism $\phi \overset{k}{\rightarrow} B : \mathcal{V}(U(A' \rightarrow B), U(A \rightarrow B))$ as

$$t : U(A' \rightarrow B) \vdash \phi \overset{k}{\rightarrow} B = \{\lambda x.x' \leftarrow \phi[\text{ret}x]; !tx'\} : U(A \rightarrow B)$$

and for $A \in \mathcal{V}$ and $f : \mathcal{V}(UB, UB')$ we define $A \overset{k}{\rightarrow} f : \mathcal{V}(U(A \rightarrow B), U(A \rightarrow B'))$ as

$$t : U(A \rightarrow B) \vdash A \overset{k}{\rightarrow} f = \{\lambda x. !f[\{!tx\}]\}$$

2. For $\overset{k}{\times}$ we define for $\phi : \mathcal{E}(FA_1, FA_2)$ and $A' \in \mathcal{V}$ the morphism $\phi \overset{k}{\times} A_2$ as

$$\bullet : F(A_1 \times A_2) \vdash \phi \overset{k}{\times} A_2 = (x_1, x_2) \leftarrow \bullet; x'_1 \leftarrow \phi[\text{ret}x_1]; \text{ret}(x'_1, x_2) : F(A'_1 \times A_2)$$

and $A_1 \overset{k}{\times} \phi$ is defined symmetrically.

3. For $\overset{k}{U}$ we need to define for $f : \mathcal{V}(UB, UB')$ a morphism $\overset{k}{U} f : \mathcal{E}(FUB, FUB')$. This is simply given by the functorial action of F : $\overset{k}{U} f = F(f)$
4. Similarly $\overset{k}{F} \phi = U\phi$

Functoriality in each argument is easily established, meaning for example for the function type is functorial in each argument:

1. $(\phi \circ \phi') \overset{k}{\rightarrow} B = (\phi' \overset{k}{\rightarrow} B) \circ (\phi \overset{k}{\rightarrow} B)$
2. $\text{id} \overset{k}{\rightarrow} B = \text{id}$
3. $A \overset{k}{\rightarrow} (f \circ f') = (A \overset{k}{\rightarrow} f) \circ (A \overset{k}{\rightarrow} f')$
4. $A \overset{k}{\rightarrow} \text{id} = \text{id}$

Finally, note that all of these constructions lift to squares in a double CBPV model since the squares themselves form a CBPV model and the projection functions preserve CBPV structure. For instance, given a square $\alpha : \phi \sqsubseteq_{Fc_o}^{Fc_i} \phi'$ and a horizontal morphism $d : B \multimap B'$ of appropriate type, we get a square

$$\alpha \overset{k}{\rightarrow} d : \phi \overset{k}{\rightarrow} B \sqsubseteq_{U(c_o \rightarrow d)}^{U(c_i \rightarrow d)} \phi' \overset{k}{\rightarrow} B'$$